

KAPITEL 3

In Python integrierte Datenstrukturen, Funktionen und Dateien

In diesem Kapitel diskutieren wir die in die Sprache Python integrierten Eigenschaften und Fähigkeiten, die uns durch das ganze Buch begleiten werden. Zusatzbibliotheken wie pandas und NumPy erweitern zwar die Rechenfunktionalität für größere Datensätze, werden aber im Prinzip gemeinsam mit den Python-eigenen Manipulationswerkzeugen eingesetzt.

Wir beginnen mit den Arbeitstieren unter Pythons Datenstrukturen: Tupel, Listen, Dictionarys und Mengen. Anschließend diskutieren wir, wie Sie Ihre eigenen wiederverwendbaren Python-Funktionen anlegen. Und schließlich schauen wir uns die Geheimnisse hinter Python-Dateiobjekten und der Interaktion mit Ihrer lokalen Festplatte an.

3.1 Datenstrukturen und Sequenzen

Pythons Datenstrukturen sind einfach, aber mächtig. Um ein kompetenter Python-Programmierer zu werden, ist es unerlässlich, ihren Einsatz zu meistern.

Tupel

Ein Tupel ist eine unveränderliche Sequenz fester Länge aus Python-Objekten. Am einfachsten lässt sich ein Tupel aus einer kommaseparierten Abfolge von Werten erzeugen:

```
In [1]: tup = 4, 5, 6
```

```
In [2]: tup
```

```
Out[2]: (4, 5, 6)
```

Wenn Sie Tupel in komplizierteren Ausdrücken definieren, ist es oft notwendig, die Werte in Klammern einzuschließen – wie in diesem Beispiel, in dem ein Tupel aus Tupeln erzeugt wird:

```
In [3]: nested_tup = (4, 5, 6), (7, 8)
```

```
In [4]: nested_tup
```

```
Out[4]: ((4, 5, 6), (7, 8))
```

Sie können alle Sequenzen oder Iteratoren in ein Tupel umwandeln, indem Sie `tuple` aufrufen:

```
In [5]: tuple([4, 0, 2])
```

```
Out[5]: (4, 0, 2)
```

```
In [6]: tup = tuple('string')
```

```
In [7]: tup
```

```
Out[7]: ('s', 't', 'r', 'i', 'n', 'g')
```

Auf die Elemente können Sie wie bei den meisten anderen Sequenztypen mit eckigen Klammern `[]` zugreifen. Wie in C, C++, Java und vielen weiteren Sprachen beginnen Sequenzen in Python mit dem Indexwert 0:

```
In [8]: tup[0]
```

```
Out[8]: 's'
```

Die Objekte, die in einem Tupel gespeichert sind, können zwar selbst veränderlich sein, doch sobald das Tupel erzeugt wurde, ist es nicht mehr möglich, zu ändern, welches Objekt in welchem Slot gespeichert ist:

```
In [9]: tup = tuple(['foo', [1, 2], True])
```

```
In [10]: tup[2] = False
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-10-c7308343b841> in <module>()
----> 1 tup[2] = False
TypeError: 'tuple' object does not support item assignment
```

Falls ein Objekt in einem Tupel veränderlich ist, wie etwa eine Liste, können Sie es an Ort und Stelle modifizieren:

```
In [11]: tup[1].append(3)
```

```
In [12]: tup
```

```
Out[12]: ('foo', [1, 2, 3], True)
```

Mit dem `+`-Operator lassen sich Tupel verketten, um längere Tupel herzustellen:

```
In [13]: (4, None, 'foo') + (6, 0) + ('bar',)
```

```
Out[13]: (4, None, 'foo', 6, 0, 'bar')
```

Wenn Sie Tupel mit einem Integer-Wert multiplizieren, hat das bei Listen den Effekt, dass entsprechend viele Kopien des Tupels miteinander verkettet werden:

```
In [14]: ('foo', 'bar') * 4
```

```
Out[14]: ('foo', 'bar', 'foo', 'bar', 'foo', 'bar', 'foo', 'bar')
```

Beachten Sie, dass die Objekte selbst nicht kopiert werden, sondern nur die Referenzen darauf.

Tupel entpacken

Falls Sie versuchen, einem tupelartigen Ausdruck oder einer Variablen etwas *zuzuweisen*, versucht Python, den Wert auf der rechten Seite des Gleichheitszeichens zu *entpacken*:

```
In [15]: tup = (4, 5, 6)
```

```
In [16]: a, b, c = tup
```

```
In [17]: b
```

```
Out[17]: 5
```

Selbst Sequenzen mit geschachtelten Tupeln können entpackt werden:

```
In [18]: tup = 4, 5, (6, 7)
```

```
In [19]: a, b, (c, d) = tup
```

```
In [20]: d
```

```
Out[20]: 7
```

Mithilfe dieser Funktionalität lassen sich ganz einfach Variablennamen tauschen – eine Aufgabe, die in vielen Sprachen folgendermaßen aussehen könnte:

```
tmp = a
```

```
a = b
```

```
b = tmp
```

In Python dagegen kann der Tausch so erfolgen:

```
In [21]: a, b = 1, 2
```

```
In [22]: a
```

```
Out[22]: 1
```

```
In [23]: b
```

```
Out[23]: 2
```

```
In [24]: b, a = a, b
```

```
In [25]: a
```

```
Out[25]: 2
```

```
In [26]: b
```

```
Out[26]: 1
```

Ein üblicher Einsatz des Entpackens von Variablen ist das Iterieren über Sequenzen von Tupeln oder Listen:

```
In [27]: seq = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

```
In [28]: for a, b, c in seq:
```

```
.....:     print('a={0}, b={1}, c={2}'.format(a, b, c))
```

```
a=1, b=2, c=3
```

```
a=4, b=5, c=6
```

```
a=7, b=8, c=9
```

Ein weiterer verbreiteter Einsatz ist die Rückgabe mehrerer Werte aus einer Funktion. Ich werde später näher darauf eingehen.

In die Sprache Python wurde vor Kurzem eine etwas raffiniertere Methode des Entpackens von Tupeln aufgenommen, die Ihnen helfen könnte, wenn Sie Elemente vom Anfang eines Tupels »herauspicken« wollen. Sie nutzt die spezielle Syntax `*rest`, die auch in Funktionssignaturen verwendet wird, um eine beliebig lange Liste von Positionsargumenten festzuhalten:

```
In [29]: values = 1, 2, 3, 4, 5
```

```
In [30]: a, b, *rest = values
```

```
In [31]: a, b  
Out[31]: (1, 2)
```

```
In [32]: rest  
Out[32]: [3, 4, 5]
```

Dieser `rest`-Teil ist manchmal etwas, das Sie loswerden wollen; es ist nichts Besonderes an dem Namen `rest`. Viele Python-Programmierer nutzen üblicherweise den Unterstrich (`_`) für unerwünschte Variablen:

```
In [33]: a, b, *_ = values
```

Methoden für Tupel

Da Größe und Inhalt eines Tupels nicht modifiziert werden können, gibt es nicht viele Instanzmethoden. Eine besonders nützliche (die es auch für Listen gibt) ist `count`, die zählt, wie oft ein Wert auftritt:

```
In [34]: a = (1, 2, 2, 2, 3, 4, 2)
```

```
In [35]: a.count(2)  
Out[35]: 4
```

Listen

Im Gegensatz zu Tupeln haben Listen eine variable Länge, und ihr Inhalt kann verändert werden. Sie können sie mithilfe von eckigen Klammern `[]` oder mit der Funktion `list` definieren:

```
In [36]: a_list = [2, 3, 7, None]
```

```
In [37]: tup = ('foo', 'bar', 'baz')
```

```
In [38]: b_list = list(tup)
```

```
In [39]: b_list  
Out[39]: ['foo', 'bar', 'baz']
```

```
In [40]: b_list[1] = 'peekaboo'
```

```
In [41]: b_list
Out[41]: ['foo', 'peekaboo', 'baz']
```

Listen und Tupel sind semantisch ähnlich (Tupel lassen sich allerdings nicht modifizieren) und können in vielen Funktionen synonym verwendet werden.

Die list-Funktion dient in der Datenverarbeitung oft als Möglichkeit, einen Iterator- oder Generator-Ausdruck einzusetzen:

```
In [42]: gen = range(10)

In [43]: gen
Out[43]: range(0, 10)

In [44]: list(gen)
Out[44]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Elemente hinzufügen und entfernen

Man kann am Ende der Liste mit der Methode `append` Elemente hinzufügen:

```
In [45]: b_list.append('dwarf')

In [46]: b_list
Out[46]: ['foo', 'peekaboo', 'baz', 'dwarf']
```

Die Methode `insert` erlaubt es Ihnen, ein Element an einer bestimmten Stelle in der Liste einzusetzen:

```
In [47]: b_list.insert(1, 'red')

In [48]: b_list
Out[48]: ['foo', 'red', 'peekaboo', 'baz', 'dwarf']
```

Der Indexwert für die Einfügung muss zwischen 0 und der Länge der Liste (einschließlich dieses Werts) liegen.



`insert` ist im Vergleich zu `append` rechnerisch aufwendig, da Verweise auf nachfolgende Elemente intern verschoben werden müssen, um Platz für das neue Element zu machen. Falls Sie sowohl am Anfang als auch am Ende einer Sequenz Elemente einfügen müssen, sollten Sie sich einmal `collections.deque` anschauen, eine Warteschlange, an der an beiden Enden angefügt werden kann (eine »double-ended queue«).

Die entgegengesetzte Operation zu `insert` ist `pop`, die ein Element an einem bestimmten Index entfernt und zurückgibt:

```
In [49]: b_list.pop(2)
Out[49]: 'peekaboo'

In [50]: b_list
Out[50]: ['foo', 'red', 'baz', 'dwarf']
```

Elemente können über ihren Wert mit der Methode `remove` entfernt werden, die den ersten entsprechenden Werte sucht und aus der Liste löscht:

```
In [51]: b_list.append('foo')

In [52]: b_list
Out[52]: ['foo', 'red', 'baz', 'dwarf', 'foo']

In [53]: b_list.remove('foo')

In [54]: b_list
Out[54]: ['red', 'baz', 'dwarf', 'foo']
```

Wenn die Leistung keine Rolle spielt, können Python-Listen mithilfe von `append` und `remove` hervorragend als Datenstruktur für Mengen mit mehrfachem Vorkommen dienen.

Mit dem Schlüsselwort `in` prüfen Sie, ob ein bestimmtes Element in der Liste auftritt:

```
In [55]: 'dwarf' in b_list
Out[55]: True
```

Das Schlüsselwort `not` negiert `in`:

```
In [56]: 'dwarf' not in b_list
Out[56]: False
```

Die Prüfung, ob eine Liste einen Wert enthält, dauert viel länger als mit Dictionaries und Sets (dazu kommen wir bald), da Python einen linearen Scan über die Werte der Liste ausführt, während es die anderen (auf der Grundlage von Hash-Tabellen) in konstanter Zeit prüfen kann.

Listen verketteten und kombinieren

Ähnlich wie Tupel lassen sich zwei Listen mit dem `+`-Operator verketteten:

```
In [57]: [4, None, 'foo'] + [7, 8, (2, 3)]
Out[57]: [4, None, 'foo', 7, 8, (2, 3)]
```

Sollten Sie die Liste bereits definiert haben, können Sie mit der Methode `extend` mehrere Elemente an sie anhängen:

```
In [58]: x = [4, None, 'foo']

In [59]: x.extend([7, 8, (2, 3)])
```

```
In [60]: x
Out[60]: [4, None, 'foo', 7, 8, (2, 3)]
```

Beachten Sie, dass die Verkettung durch Addition eine vergleichsweise aufwendige Operation ist, da eine neue Liste angelegt werden muss und die Objekte dann hinüberkopiert werden müssen. Besser ist es, wenn Sie `extend` nutzen, um Elemente an eine vorhandene Liste anzuhängen, vor allem wenn Sie eine sehr große Liste erstellen.

Daher ist

```
everything = []
for chunk in list_of_lists:
    everything.extend(chunk)
```

schneller als die Alternative mit der Verkettung:

```
everything = []
for chunk in list_of_lists:
    everything = everything + chunk
```

Sortieren

Sie können eine Liste an Ort und Stelle sortieren (ohne ein neues Objekt anzulegen), indem Sie ihre Funktion `sort` aufrufen:

```
In [61]: a = [7, 2, 5, 1, 3]
```

```
In [62]: a.sort()
```

```
In [63]: a
```

```
Out[63]: [1, 2, 3, 5, 7]
```

`sort` besitzt einige Optionen, die gelegentlich ganz praktisch sind. Eine ist die Fähigkeit, einen sekundären *Sortierschlüssel* zu übergeben – das heißt eine Funktion, die einen Wert zum Sortieren der Objekte erzeugt. Zum Beispiel könnten wir eine Kollektion von Strings nach ihrer Länge sortieren:

```
In [64]: b = ['saw', 'small', 'He', 'foxes', 'six']
```

```
In [65]: b.sort(key=len)
```

```
In [66]: b
```

```
Out[66]: ['He', 'saw', 'six', 'small', 'foxes']
```

Wir werden uns gleich die `sorted`-Funktion anschauen, die eine sortierte Kopie einer allgemeinen Sequenz erzeugen kann.

Binärsuche und Aufrechterhalten einer sortierten Liste

Das integrierte `bisect`-Modul implementiert die Binärsuche und das Einfügen in eine sortierte Liste. `bisect.bisect` sucht den Ort, an dem ein Element eingefügt werden müsste, um die Sortierung aufrechtzuerhalten, während `bisect.insort` das Element tatsächlich an diesem Ort einfügt:

```
In [67]: import bisect
```

```
In [68]: c = [1, 2, 2, 2, 3, 4, 7]
```

```
In [69]: bisect.bisect(c, 2)
```

```
Out[69]: 4
```

```
In [70]: bisect.bisect(c, 5)
```

```
Out[70]: 6
```

```
In [71]: bisect.insort(c, 6)

In [72]: c
Out[72]: [1, 2, 2, 2, 3, 4, 6, 7]
```



Die `bisect`-Modulfunktionen prüfen nicht, ob die Liste sortiert ist, da dies rechnerisch sehr aufwendig wäre. Daher liefert ihr Einsatz an einer unsortierten Liste keine Fehlermeldung, allerdings könnten die Ergebnisse fehlerhaft sein.

Slicing

Sie können Abschnitte von listenähnlichen Typen mit der Slice-Notation auswählen. Diese besteht in ihrer grundlegendsten Form aus dem Intervall `start:stop`, das dem Indexoperator `[]` übergeben wird:

```
In [73]: seq = [7, 2, 3, 7, 5, 6, 0, 1]

In [74]: seq[1:5]
Out[74]: [2, 3, 7, 5]
```

An Slices lassen sich mit einer Sequenz auch Zuweisungen vornehmen:

```
In [75]: seq[3:4] = [6, 3]

In [76]: seq
Out[76]: [7, 2, 3, 6, 3, 5, 6, 0, 1]
```

Das Element am Index `start` ist enthalten, während das am `stop`-Index *nicht enthalten* ist. Die Anzahl der Elemente im Ergebnis ist daher genau `stop - start`.

Entweder `start` oder `stop` kann weggelassen werden. In diesem Fall wird standardmäßig der Beginn bzw. das Ende der Sequenz angenommen:

```
In [77]: seq[:5]
Out[77]: [7, 2, 3, 6, 3]

In [78]: seq[3:]
Out[78]: [6, 3, 5, 6, 0, 1]
```

Negative Indizes erzeugen einen Abschnitt relativ zum Ende:

```
In [79]: seq[-4:]
Out[79]: [5, 6, 0, 1]

In [80]: seq[-6:-2]
Out[80]: [6, 3, 5, 6]
```

Die Slicing-Semantik ist ein bisschen gewöhnungsbedürftig, vor allem wenn Sie von R oder MATLAB kommen. In Abbildung 3-1 finden Sie eine hilfreiche Darstellung des Slicings mit positiven und negativen Indizes. In der Abbildung sind die Indizes klein gedruckt in den Ecken mit angegeben, damit Sie erkennen, wo eine Slice-Auswahl bei positiven und negativen Indexwerten beginnt und endet.

Nach einem zweiten Doppelpunkt kann ein `step` angegeben werden, um z. B. jedes zweite Element auszuwählen:

```
In [81]: seq[::2]
Out[81]: [7, 3, 3, 6, 1]
```

Eine sinnvolle Anwendung dieser Option wäre etwa, `-1` zu übergeben. Damit lässt sich nämlich eine Liste oder ein Tupel umkehren:

```
In [82]: seq[::-1]
Out[82]: [1, 0, 6, 5, 3, 6, 3, 2, 7]
```

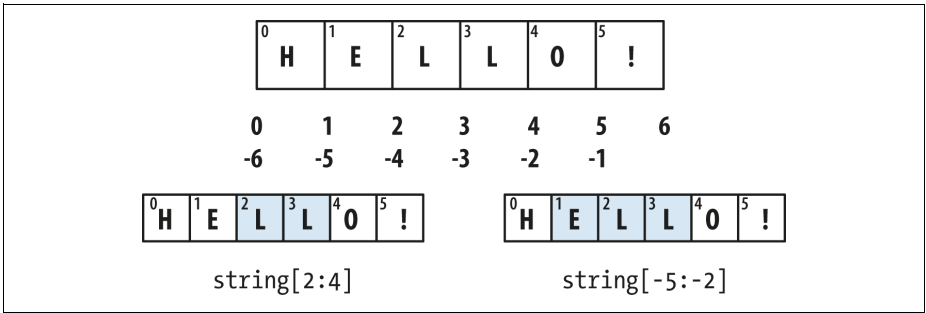


Abbildung 3-1: Darstellung der Slicing-Konventionen von Python

Eingebaute Funktionen von Sequenzen

Python besitzt eine Reihe von nützlichen Sequenzfunktionen, mit denen Sie sich vertraut machen und die Sie so oft wie möglich einsetzen sollten.

enumerate

Beim Iterieren über eine Sequenz möchte man oft den Index des aktuellen Elements mitführen. Ein selbst gemachter Ansatz dafür sähe folgendermaßen aus:

```
i = 0
for value in collection:
    # do something with value
    i += 1
```

Da das so häufig vorkommt, besitzt Python dafür eine Funktion, `enumerate`, die eine Sequenz von `(i, value)`-Tupeln generiert:

```
for i, value in enumerate(collection):
    # do something with value
```

Ein sinnvolles Vorgehen beim Indizieren von Daten ist, mit `enumerate` ein dict zu erzeugen, das die Werte einer Sequenz (von denen man annimmt, dass sie eindeutig sind) auf ihre Plätze in der Sequenz abbildet:

```
In [83]: some_list = ['foo', 'bar', 'baz']
```

```
In [84]: mapping = {}
```

```
In [85]: for i, v in enumerate(some_list):
....:     mapping[v] = i
```

```
In [86]: mapping
Out[86]: {'bar': 1, 'baz': 2, 'foo': 0}
```

sorted

Die Funktion `sorted` liefert eine neue sortierte Liste aus den Elementen einer Sequenz zurück:

```
In [87]: sorted([7, 1, 2, 6, 0, 3, 2])
Out[87]: [0, 1, 2, 2, 3, 6, 7]
```

```
In [88]: sorted('horse race')
Out[88]: [' ', 'a', 'c', 'e', 'e', 'h', 'o', 'r', 'r', 's']
```

`sorted` akzeptiert die gleichen Argumente wie die Methode `sort` bei Listen.

zip

`zip` ordnet die Elemente von Listen, Tupeln oder anderen Sequenzen paarweise an und generiert so eine Liste aus Tupeln:

```
In [89]: seq1 = ['foo', 'bar', 'baz']
```

```
In [90]: seq2 = ['one', 'two', 'three']
```

```
In [91]: zipped = zip(seq1, seq2)
```

```
In [92]: list(zipped)
```

```
Out[92]: [('foo', 'one'), ('bar', 'two'), ('baz', 'three')]
```

`zip` kann eine beliebige Anzahl von Sequenzen entgegennehmen. Die Anzahl der generierten Elemente wird durch die *kürzeste* Sequenz bestimmt:

```
In [93]: seq3 = [False, True]
```

```
In [94]: list(zip(seq1, seq2, seq3))
```

```
Out[94]: [('foo', 'one', False), ('bar', 'two', True)]
```

Ein verbreiteter Einsatz von `zip` ist das gleichzeitige Iterieren über mehrere Sequenzen, möglicherweise in Kombination mit `enumerate`:

```
In [95]: for i, (a, b) in enumerate(zip(seq1, seq2)):
....:     print('{0}: {1}, {2}'.format(i, a, b))
....:
0: foo, one
1: bar, two
2: baz, three
```

Man kann `zip` ziemlich clever auf eine »gezippte« Sequenz anwenden, um sie zu »entzippen«. Das Ganze lässt sich auch als Umwandlung einer Liste aus *Zeilen* in eine Liste aus *Spalten* ansehen. Die durchaus magisch wirkende Syntax sieht so aus:

```
In [96]: pitchers = [('Nolan', 'Ryan'), ('Roger', 'Clemens'),
....:                ('Curt', 'Schilling')]

In [97]: first_names, last_names = zip(*pitchers)

In [98]: first_names
Out[98]: ('Nolan', 'Roger', 'Curt')

In [99]: last_names
Out[99]: ('Ryan', 'Clemens', 'Schilling')
```

reversed

`reversed` iteriert in umgekehrter Reihenfolge über die Elemente einer Sequenz:

```
In [100]: list(reversed(range(10)))
Out[100]: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Denken Sie daran, dass `reversed` ein Generator ist (darauf kommen wir später genauer zurück), es erzeugt die umgekehrte Sequenz also nur, wenn schon vorher eine Sequenz erzeugt wurde (z.B. mit `list` oder einer `for`-Schleife).

Dictionarys

Dictionarys (`dict`) gelten als die wichtigste in Python eingebaute Datenstruktur. Ein gebräuchlicherer Name dafür ist *Hash-Tabelle* oder *assoziatives Array*. Dabei handelt es sich um eine Kollektion flexibler GröÙe aus Schlüssel-Wert-Paaren, wobei *Schlüssel* und *Wert* Python-Objekte sind. Eine Möglichkeit, sie herzustellen, besteht im Einsatz von geschweiften Klammern `{}` und Doppelpunkten zum Trennen der Schlüssel von den Werten:

```
In [101]: empty_dict = {}

In [102]: d1 = {'a' : 'some value', 'b' : [1, 2, 3, 4]}

In [103]: d1
Out[103]: {'a': 'some value', 'b': [1, 2, 3, 4]}
```

Zum Zugreifen, Einfügen oder Setzen von Elementen nutzen Sie die gleiche Syntax wie beim Zugreifen von Elementen einer Liste oder eines Tupels:

```
In [104]: d1[7] = 'an integer'

In [105]: d1
Out[105]: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'an integer'}

In [106]: d1['b']
Out[106]: [1, 2, 3, 4]
```

Um zu prüfen, ob ein Dictionary einen bestimmten Schlüssel enthält, verwenden Sie die gleiche Syntax, mit der Sie prüfen, ob eine Liste oder ein Tupel einen bestimmten Wert enthält:

```
In [107]: 'b' in d1
Out[107]: True
```

Sie können Werte entweder mit dem Schlüsselwort `del` oder der Methode `pop` löschen (die gleichzeitig den Wert zurückgibt und den Schlüssel löscht):

```
In [108]: d1[5] = 'some value'
```

```
In [109]: d1
Out[109]:
{'a': 'some value',
 'b': [1, 2, 3, 4],
 7: 'an integer',
 5: 'some value'}
```

```
In [110]: d1['dummy'] = 'another value'
```

```
In [111]: d1
Out[111]:
{'a': 'some value',
 'b': [1, 2, 3, 4],
 7: 'an integer',
 5: 'some value',
 'dummy': 'another value'}
```

```
In [112]: del d1[5]
```

```
In [113]: d1
Out[113]:
{'a': 'some value',
 'b': [1, 2, 3, 4],
 7: 'an integer',
 'dummy': 'another value'}
```

```
In [114]: ret = d1.pop('dummy')
```

```
In [115]: ret
Out[115]: 'another value'
```

```
In [116]: d1
Out[116]: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'an integer'}
```

Die `keys`- und `values`-Methoden liefern Ihnen Iteratoren der Schlüssel bzw. Werte des Dictionarys. Während die Schlüssel-Wert-Paare nicht in einer bestimmten Reihenfolge vorliegen, geben diese Funktionen die Schlüssel und Werte in der gleichen Reihenfolge aus:

```
In [117]: list(d1.keys())
Out[117]: ['a', 'b', 7]
```

```
In [118]: list(d1.values())
Out[118]: ['some value', [1, 2, 3, 4], 'an integer']
```

Die Methode `update` erlaubt Ihnen, ein Dictionary in ein anderes zu übernehmen:

```
In [119]: d1.update({'b' : 'foo', 'c' : 12})
```

```
In [120]: d1
```

```
Out[120]: {'a': 'some value', 'b': 'foo', 7: 'an integer', 'c': 12}
```

`update` ändert Dictionarys an Ort und Stelle. Werden also Schlüssel übergeben, die es bereits gibt, werden deren alte Werte verworfen und durch die neuen mittels `update` übergebenen Daten ersetzt.

Dictionarys aus Sequenzen erzeugen

Oft passiert es, dass man zwei Sequenzen hat, deren Elemente man paarweise in einem Dictionary zusammenfassen möchte. Im ersten Anlauf könnte man folgenden Code dafür schreiben:

```
mapping = {}
for key, value in zip(key_list, value_list):
    mapping[key] = value
```

Da ein Dictionary im Prinzip eine Kollektion aus 2-Tupeln ist, akzeptiert die `dict`-Funktion eine Liste aus 2-Tupeln:

```
In [121]: mapping = dict(zip(range(5), reversed(range(5))))
```

```
In [122]: mapping
```

```
Out[122]: {0: 4, 1: 3, 2: 2, 3: 1, 4: 0}
```

Später werden wir noch über *Dict Comprehensions* sprechen, eine weitere elegante Möglichkeit, Dictionarys zu konstruieren.

Voreingestellte Werte

Oft findet man eine solche logische Abfolge vor:

```
if key in some_dict:
    value = some_dict[key]
else:
    value = default_value
```

Das bedeutet, die Methoden `get` und `pop` liefern unter Umständen einen voreingestellten Wert zurück, sodass sich der oben gezeigte `if-else`-Block auch einfach so ausdrücken lässt:

```
value = some_dict.get(key, default_value)
```

`get` liefert standardmäßig `None`, falls es keinen Schlüssel gibt, während `pop` einen Ausnahmefehler erzeugt. Beim *Setzen* von Werten sind die Werte in einem Dictionary häufig weitere Kollektionen, wie etwa Listen. So könnte man zum Beispiel eine Wortliste anhand ihrer Anfangsbuchstaben als ein Dictionary von Listen kategorisieren:

```
In [123]: words = ['apple', 'bat', 'bar', 'atom', 'book']
```

```
In [124]: by_letter = {}
```

```
In [125]: for word in words:
.....:     letter = word[0]
.....:     if letter not in by_letter:
.....:         by_letter[letter] = [word]
.....:     else:
.....:         by_letter[letter].append(word)
.....:

In [126]: by_letter
Out[126]: {'a': ['apple', 'atom'], 'b': ['bat', 'bar', 'book']}
```

Die Methode `setdefault` dient genau diesem Zweck. Die `for`-Schleife lässt sich so umschreiben:

```
for word in words:
    letter = word[0]
    by_letter.setdefault(letter, []).append(word)
```

Das eingebaute `collections`-Modul besitzt eine nützliche Klasse namens `defaultdict`, die dies noch weiter vereinfacht. Sie übergeben einen Typ oder eine Funktion zum Generieren des Standardwerts für jeden Eintrag des Dictionarys:

```
from collections import defaultdict
by_letter = defaultdict(list)
for word in words:
    by_letter[word[0]].append(word)
```

Zulässige Schlüsseltypen für Dictionarys

Während die Werte eines Dictionarys beliebige Python-Objekte sein können, müssen die Schlüssel im Allgemeinen unveränderliche Objekte sein, wie Skalare (Integer, Float, String) oder Tupel (auch die Objekte in dem Tupel müssen unveränderlich sein). Die technische Bezeichnung dafür ist *Hashability*. Sie können mit der Funktion `hash` prüfen, ob ein Objekt hashable ist (also als Schlüssel in einem Dictionary verwendet werden kann):

```
In [127]: hash('string')
Out[127]: 5023931463650008331
```

```
In [128]: hash((1, 2, (2, 3)))
Out[128]: 1097636502276347782
```

```
In [129]: hash((1, 2, [2, 3])) # fails because lists are mutable
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-129-800cd14ba8be> in <module>()
----> 1 hash((1, 2, [2, 3])) # fails because lists are mutable
TypeError: unhashable type: 'list'
```

Wenn Sie eine Liste als Schlüssel verwenden wollen, besteht eine Möglichkeit darin, sie in ein Tupel umzuwandeln, das hashable ist, solange seine Elemente es ebenfalls sind:

```
In [130]: d = {}  
  
In [131]: d[tuple([1, 2, 3])] = 5  
  
In [132]: d  
Out[132]: {(1, 2, 3): 5}
```

Set

Ein Set ist eine ungeordnete Kollektion eindeutiger Elemente. Sie können sie sich wie Dictionaries vorstellen, die nur aus Schlüsseln bestehen, also keine Werte enthalten. Es gibt zwei Wege, um ein Set zu erzeugen: über die Funktion `set` oder einen *set-Ausdruck* mit geschweiften Klammern:

```
In [133]: set([2, 2, 2, 1, 3, 3])  
Out[133]: {1, 2, 3}  
  
In [134]: {2, 2, 2, 1, 3, 3}  
Out[134]: {1, 2, 3}
```

Sets unterstützen mathematische *Mengenoperationen* wie Vereinigung, Schnitt, Differenz und symmetrische Differenz. Schauen Sie sich diese beiden Beispielmengen an:

```
In [135]: a = {1, 2, 3, 4, 5}  
  
In [136]: b = {3, 4, 5, 6, 7, 8}
```

Die Vereinigung dieser beiden Sets ist die Menge der eindeutigen Elemente, die in mindestens einem der beiden Sets enthalten ist. Das lässt sich mit der `union`-Methode oder dem Binäroperator `|` berechnen:

```
In [137]: a.union(b)  
Out[137]: {1, 2, 3, 4, 5, 6, 7, 8}  
  
In [138]: a | b  
Out[138]: {1, 2, 3, 4, 5, 6, 7, 8}
```

Die Schnittmenge enthält die Elemente, die in beiden Sets enthalten sind. Hier kann der `&`-Operator oder die Methode `intersection` benutzt werden:

```
In [139]: a.intersection(b)  
Out[139]: {3, 4, 5}  
  
In [140]: a & b  
Out[140]: {3, 4, 5}
```

In Tabelle 3-1 finden Sie eine Liste der gebräuchlichsten Methoden für Sets.

Tabelle 3-1: Python-Mengenoperationen

Funktion	Alternative Syntax	Beschreibung
<code>a.add(x)</code>	N/A	Fügt das Element <code>x</code> dem Set <code>a</code> hinzu.
<code>a.clear()</code>	N/A	Setzt das Set <code>a</code> in einen leeren Zustand zurück und verwirft alle seine Elemente.

Tabelle 3-1: Python-Mengenoperationen (Fortsetzung)

Funktion	Alternative Syntax	Beschreibung
a.remove(x)	N/A	Entfernt das Element x aus dem Set a.
a.pop()	N/A	Entfernt ein beliebiges Element aus dem Set a und gibt einen KeyError aus, falls das Set leer ist.
a.union(b)	a b	Alle eindeutigen Elemente in a und b.
a.update(b)	a = b	Setzt den Inhalt von a auf die Vereinigungsmenge der Elemente aus a und b.
a.intersection(b)	a & b	Alle Elemente, die sowohl in a als auch in b enthalten sind.
a.intersection_update(b)	a &= b	Setzt den Inhalt von a auf die Schnittmenge der Elemente aus a und b.
a.difference(b)	a - b	Die Elemente aus a, die nicht in b enthalten sind.
a.difference_update(b)	a -= b	Setzt a auf die Elemente aus a, die nicht in b enthalten sind.
a.symmetric_difference(b)	a ^ b	Alle Elemente in entweder a oder b, aber nicht in beiden Sets.
a.symmetric_difference_update(b)	a ^= b	Setzt a auf die Elemente, die in entweder a oder b, aber nicht in beiden Sets enthalten sind
a.issubset(b)	<=	True, falls die Elemente von a alle in b enthalten sind.
a.issuperset(b)	=>	True, falls die Elemente von b alle in a enthalten sind.
a.isdisjoint(b)	N/A	True, falls a und b keine gemeinsamen Elemente enthalten.

Alle logischen Mengenoperationen besitzen »In-Place«-Gegenstücke, die es Ihnen erlauben, den Inhalt des Sets auf der linken Seite der Operation durch das Ergebnis zu ersetzen. Bei sehr großen Sets könnte dies effizienter sein:

```
In [141]: c = a.copy()

In [142]: c |= b

In [143]: c
Out[143]: {1, 2, 3, 4, 5, 6, 7, 8}

In [144]: d = a.copy()

In [145]: d &= b

In [146]: d
Out[146]: {3, 4, 5}
```

Genau wie Dictionaries müssen Set-Elemente unveränderlich sein. Um listenartige Elemente verarbeiten zu können, müssen Sie sie in ein Tupel umwandeln:

```
In [147]: my_data = [1, 2, 3, 4]

In [148]: my_set = {tuple(my_data)}
```

```
In [149]: my_set
Out[149]: {(1, 2, 3, 4)}
```

Sie können auch prüfen, ob ein Set eine Teilmenge (das heißt enthalten ist in) oder eine Obermenge (enthält alle Elemente) eines anderen Sets ist:

```
In [150]: a_set = {1, 2, 3, 4, 5}

In [151]: {1, 2, 3}.issubset(a_set)
Out[151]: True

In [152]: a_set.issuperset({1, 2, 3})
Out[152]: True
```

Sets sind gleich, wenn und nur wenn ihre Inhalte gleich sind:

```
In [153]: {1, 2, 3} == {3, 2, 1}
Out[153]: True
```

List, Set und Dict Comprehensions

List Comprehensions gehören zu den beliebtesten Merkmalen von Python. Sie erlauben Ihnen, mit einem knappen Ausdruck eine neue Liste zu formen, indem die Elemente einer Kollektion gefiltert und die gefilterten Elemente transformiert werden. Sie haben folgende Grundform:

```
[expr for val in collection if condition]
```

Das ist äquivalent zu der folgenden for-Schleife:

```
result = []
for val in collection:
    if condition:
        result.append(expr)
```

Man kann die Filterbedingung weglassen, sodass nur der Ausdruck übrig bleibt. Zum Beispiel könnten wir aus einer Liste von Strings alle Strings mit einer maximalen Länge von 2 herausfiltern und sie gleichzeitig in Großbuchstaben konvertieren:

```
In [154]: strings = ['a', 'as', 'bat', 'car', 'dove', 'python']

In [155]: [x.upper() for x in strings if len(x) > 2]
Out[155]: ['BAT', 'CAR', 'DOVE', 'PYTHON']
```

Set und Dict Comprehensions sind eine natürliche Erweiterung. Sie erzeugen auf idiomatisch ähnliche Weise Sets und Dictionarys anstelle von Listen. Eine Dict Comprehension sehen Sie beispielsweise hier:

```
dict_comp = {key-expr : value-expr for value in collection
              if condition}
```

Eine Set Comprehension sieht wie die äquivalente List Comprehension aus, allerdings mit geschweiften statt mit eckigen Klammern:

```
set_comp = {expr for value in collection if condition}
```

Genau wie List Comprehensions dienen Set und Dict Comprehensions vor allem der Bequemlichkeit, erleichtern aber auch das Schreiben und Lesen von Code. Denken Sie an die obige Liste der Strings. Würden wir zum Beispiel nur die Längen der Strings haben wollen, die in der Kollektion enthalten sind, wäre das mit einer Set Comprehension leicht zu erreichen:

```
In [156]: unique_lengths = {len(x) for x in strings}
```

```
In [157]: unique_lengths
Out[157]: {1, 2, 3, 4, 6}
```

Wir könnten dies auch mit der map-Funktion ausdrücken, auf die wir in Kürze genauer eingehen:

```
In [158]: set(map(len, strings))
Out[158]: {1, 2, 3, 4, 6}
```

Als einfaches Beispiel für eine Dict Comprehension erstellen wir eine Tabelle, die diese Strings ihren Orten in der Liste zuordnet:

```
In [159]: loc_mapping = {val : index for index, val in enumerate(strings)}
```

```
In [160]: loc_mapping
Out[160]: {'a': 0, 'as': 1, 'bat': 2, 'car': 3, 'dove': 4, 'python': 5}
```

Verschachtelte List Comprehensions

Nehmen wir einmal an, wir haben eine Liste von Listen mit englischen und spanischen Namen:

```
In [161]: all_data = [['John', 'Emily', 'Michael', 'Mary', 'Steven'],
.....:                ['Maria', 'Juan', 'Javier', 'Natalia', 'Pilar']]
```

Sie haben diese Namen vielleicht einigen Dateien entnommen und beschlossen, sie nach der Sprache zu organisieren. Stellen Sie sich nun vor, Sie möchten eine einzige Liste haben, die alle Namen mit einem oder mehreren e enthält. Das ließe sich sicherlich mit einer einfachen for-Schleife bewerkstelligen:

```
names_of_interest = []
for names in all_data:
    enough_es = [name for name in names if name.count('e') >= 2]
    names_of_interest.extend(enough_es)
```

Sie können den ganzen Vorgang auch in einer einzigen *geschachtelten List Comprehension* zusammenfassen, die so aussieht:

```
In [162]: result = [name for names in all_data for name in names
.....:                if name.count('e') >= 2]
```

```
In [163]: result
Out[163]: ['Steven']
```

Geschachtelte List Comprehensions sind zu Beginn vielleicht etwas schwerer zu verstehen. Die `for`-Teile der List Comprehension werden entsprechend der Reihenfolge der Schachtelung angeordnet, Filterbedingungen kommen wie gehabt ans Ende. Hier ist ein weiteres Beispiel, in dem wir eine Liste aus Tupeln mit Integer-Werten in eine einfache Liste aus Integer-Werten »verflachen«:

```
In [164]: some_tuples = [(1, 2, 3), (4, 5, 6), (7, 8, 9)]
```

```
In [165]: flattened = [x for tup in some_tuples for x in tup]
```

```
In [166]: flattened
```

```
Out[166]: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Merken Sie sich, dass die Reihenfolge der `for`-Ausdrücke die gleiche wäre, wenn Sie statt einer List Comprehension eine geschachtelte `for`-Schleife geschrieben hätten:

```
flattened = []
```

```
for tup in some_tuples:
    for x in tup:
        flattened.append(x)
```

Es sind beliebig viele Schachtelungsstufen möglich, allerdings sollten Sie sich bei mehr als zwei oder drei Ineinanderschachtelungen fragen, ob das Ganze aus Sicht der Lesbarkeit des Codes noch sinnvoll ist. Es ist wichtig, die gerade gezeigte Syntax von einer List Comprehension innerhalb einer List Comprehension zu unterscheiden, die ebenfalls absolut zulässig ist:

```
In [167]: [[x for x in tup] for tup in some_tuples]
```

```
Out[167]: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Dies erzeugt eine Liste aus Listen statt einer verflachten Liste aller inneren Elemente.

3.2 Funktionen

Funktionen stellen die primäre und wichtigste Methode dar, um Code in Python zu organisieren und wiederverwendbar zu machen. Wenn Faustregel gilt: Falls Sie glauben, dass Sie denselben oder ähnlichen Code mehr als einmal brauchen werden, lohnt es sich vermutlich, eine wiederverwendbare Funktion zu schreiben. Funktionen helfen außerdem dabei, Code besser lesbar zu machen, indem sie einer Gruppe von Python-Anweisungen einen Namen geben.

Funktionen werden mit dem Schlüsselwort `def` deklariert und mit dem Schlüsselwort `return` wieder verlassen:

```
def my_function(x, y, z=1.5):
    if z > 1:
        return z * (x + y)
    else:
        return z / (x + y)
```

Es ist kein Problem, mehrere `return`-Anweisungen zu haben. Erreicht Python das Ende einer Funktion, ohne dass es eine `return`-Anweisung entdeckt, wird automatisch `None` zurückgeliefert.

Jede Funktion kann sowohl *Positions*- als auch *Keyword*-Argumente haben. Keyword-Argumente werden meist verwendet, um Standardwerte oder optionale Argumente anzugeben. In der gerade gezeigten Funktion sind `x` und `y` Positionsargumente, während `z` ein Keyword-Argument ist. Das bedeutet, dass die Funktion auf eine dieser Weisen aufgerufen werden kann:

```
my_function(5, 6, z=0.7)
my_function(3.14, 7, 3.5)
my_function(10, 20)
```

Die Haupteinschränkung bei Funktionsargumenten besteht darin, dass die Keyword-Argumente den Positionsargumenten folgen *müssen* (sollten welche vorhanden sein). Keyword-Argumente können Sie in beliebiger Reihenfolge festlegen. Sie müssen sich also nicht merken, in welcher Reihenfolge die Funktionsargumente angegeben wurden, sondern lediglich, welche Namen sie tragen.



Es ist möglich, Keywords zum Übergeben von Positionsargumenten zu verwenden. Im vorangegangenen Beispiel hätten wir auch schreiben können:

```
my_function(x=5, y=6, z=7)
my_function(y=6, x=5, z=7)
```

Manchmal hilft das bei der Lesbarkeit.

Namensraum, Gültigkeitsbereich und lokale Funktionen

Funktionen können in zwei unterschiedlichen Gültigkeitsbereichen auf Variablen zugreifen: *global* und *lokal*. Ein alternativer und aussagekräftigerer Name zum Beschreiben des Gültigkeitsbereichs von Variablen in Python ist *Namensraum*. Alle Variablen, die innerhalb einer Funktion zugewiesen werden, gehören standardmäßig zum lokalen Namensraum. Der lokale Namensraum wird beim Aufruf einer Funktion erzeugt und sofort mit den Argumenten der Funktion gefüllt. Nach dem Beenden der Funktion wird der lokale Namensraum zerstört (mit einigen Ausnahmen, die aber hier außer Acht gelassen werden sollen). Schauen Sie sich die folgende Funktion an:

```
def func():
    a = []
    for i in range(5):
        a.append(i)
```

Wird `func()` aufgerufen, wird die leere Liste `a` erzeugt, fünf Elemente werden angehängt, und anschließend wird `a` zerstört, wenn die Funktion beendet wird. Nehmen wir an, wir hätten `a` folgendermaßen deklariert:

```
a = []
def func():
```

```
for i in range(5):
    a.append(i)
```

Jeder Aufruf von `func` modifiziert die Liste `a`:

```
func()
a
func()
a
```

Es ist möglich, Variablen außerhalb des Gültigkeitsbereichs der Funktion zuzuweisen, allerdings müssen diese Variablen dann über das Schlüsselwort `global` als `global` deklariert werden:

```
In [168]: a = None

In [169]: def bind_a_variable():
.....:     global a
.....:     a = []
.....:     bind_a_variable()
.....:

In [170]: print(a)
[]
```



Ich rate im Allgemeinen von der Verwendung des Schlüsselworts `global` ab. Üblicherweise werden globale Variablen verwendet, um irgendeine Art von Zustand in einem System zu speichern. Wenn Sie feststellen, dass Sie viele davon einsetzen, sollten Sie vielleicht in die objektorientierte Programmierung (mit Klassen) wechseln.

Mehrere Rückgabewerte

Als ich nach Java und C++ das erste Mal in Python programmierte, war eine meiner Lieblingseigenschaften die Fähigkeit, mit einer einfachen Syntax mehrere Werte aus einer Funktion zurückzugeben. Hier ist ein Beispiel:

```
def f():
    a = 5
    b = 6
    c = 7
    return a, b, c

a, b, c = f()
```

In der Datenanalyse und anderen wissenschaftlichen Anwendungen werden Sie das möglicherweise recht oft tun. Hier geschieht Folgendes: Die Funktion gibt eigentlich nur ein Objekt zurück, nämlich ein Tupel, das dann in die Ergebnisvariablen entpackt wird. Im gezeigten Beispiel hätten wir stattdessen auch das machen können:

```
return_value = f()
```

In diesem Fall wäre `return_value` ein 3-Tupel mit den drei zurückgegebenen Variablen. Eine potenziell attraktive Alternative zu mehreren Rückgabewerten könnte das Zurückliefern eines Dictionarys sein:

```
def f():
    a = 5
    b = 6
    c = 7
    return {'a' : a, 'b' : b, 'c' : c}
```

Je nachdem, was Sie machen wollen, kann diese alternative Technik ganz sinnvoll sein.

Funktionen sind Objekte

Da Python-Funktionen Objekte sind, lassen sich viele in anderen Sprachen schwierige Konstrukte hier leicht ausdrücken. Nehmen wir einmal an, wir würden einige Daten aufräumen wollen und müssen auf die folgende Liste aus Strings einige Transformationen anwenden:

```
In [171]: states = [' Alabama ', 'Georgia!', 'Georgia', 'georgia', 'Fl0rIda',
.....:             'south carolina##', 'West virginia?']
```

Jeder, der schon einmal mit Umfragedaten gearbeitet hat, die von den Anwendern geliefert wurden, kennt solche chaotischen Ergebnisse. Hier muss einiges passieren, um aus dieser Liste aus Strings ein einheitliches Ganzes zu formen, das sich für eine Analyse eignet: das Entfernen von Whitespace (Leerräumen), das Löschen von Interpunktionszeichen und die Vereinheitlichung der Groß- und Kleinschreibung. Eine Möglichkeit dazu bieten die eingebauten String-Methoden zusammen mit dem Standardbibliotheksmodul `re` für reguläre Ausdrücke:

```
import re

def clean_strings(strings):
    result = []
    for value in strings:
        value = value.strip()
        value = re.sub('[!#?]', '', value)
        value = value.title()
        result.append(value)
    return result
```

Das Ergebnis sieht so aus:

```
In [173]: clean_strings(states)
Out[173]:
['Alabama',
'Georgia',
'Georgia',
'Georgia',
'Florida',
'South Carolina',
'West Virginia']
```

Ein nützliches alternatives Vorgehen besteht darin, eine Liste der Operationen anzulegen, die Sie auf einen speziellen Satz aus Strings anwenden wollen:

```
def remove_punctuation(value):
    return re.sub('[!#?]', '', value)

clean_ops = [str.strip, remove_punctuation, str.title]

def clean_strings(strings, ops):
    result = []
    for value in strings:
        for function in ops:
            value = function(value)
        result.append(value)
    return result
```

Wir erhalten Folgendes:

```
In [175]: clean_strings(states, clean_ops)
Out[175]:
['Alabama',
 'Georgia',
 'Georgia',
 'Georgia',
 'Florida',
 'South Carolina',
 'West Virginia']
```

Ein stärker *funktionales* Muster wie dieses erlaubt Ihnen, auf einer sehr abstrakten Ebene zu modifizieren, wie die Strings transformiert werden sollen. Die Funktion `clean_strings` ist nun ebenfalls leichter wiederverwendbar.

Sie können Funktionen als Argumente für andere Funktionen einsetzen, wie die eingebaute `map`-Funktion, die eine Funktion auf eine Sequenz irgendeiner Art anwendet:

```
In [176]: for x in map(remove_punctuation, states):
.....:     print(x)
Alabama
Georgia
Georgia
georgia
FlOrIda
south carolina
West virginia
```

Anonyme oder Lambda-Funktionen

Python unterstützt sogenannte *anonyme* oder *Lambda*-Funktionen. Diese bilden eine Möglichkeit, Funktionen zu schreiben, die aus einer einzigen Anweisung bestehen, deren Ergebnis der Rückgabewert ist. Sie werden mit dem Schlüsselwort `lambda` definiert, das keine andere Bedeutung hat als »wir deklarieren eine anonyme Funktion«:

```
def short_function(x):
    return x * 2
```

```
equiv_anon = lambda x: x * 2
```

Ich werde diese Funktionen im weiteren Verlauf des Buchs als Lambda-Funktionen bezeichnen. Sie sind in der Datenanalyse besonders praktisch, weil es, wie Sie noch sehen werden, viele Fälle gibt, in denen Funktionen zur Datentransformation Funktionen als Argumente erwarten. Oft bedeutet es weniger Tipparbeit (und ist auch ordentlicher), eine Lambda-Funktion zu übergeben, als eine ausgewachsene Funktionsdeklaration zu schreiben oder gar die Lambda-Funktion einer lokalen Variablen zuzuweisen. Schauen Sie sich zum Beispiel dieses alberne Beispiel an:

```
def apply_to_list(some_list, f):
    return [f(x) for x in some_list]

ints = [4, 0, 1, 5, 6]
apply_to_list(ints, lambda x: x * 2)
```

Sie hätten auch `[x * 2 for x in ints]` schreiben können, doch hier konnten wir ganz kurz und knapp einen benutzerdefinierten Operator an die Funktion `apply_to_list` übergeben.

Ein weiteres Beispiel: Stellen Sie sich vor, Sie wollten eine Kollektion aus Strings nach der Anzahl der unterschiedlichen Buchstaben in den einzelnen Strings sortieren:

```
In [177]: strings = ['foo', 'card', 'bar', 'aaaa', 'abab']
```

Wir könnten hier eine Lambda-Funktion an die `sort`-Methode der Liste übergeben:

```
In [178]: strings.sort(key=lambda x: len(set(list(x))))

In [179]: strings
Out[179]: ['aaaa', 'foo', 'abab', 'bar', 'card']
```



Lambda-Funktionen werden unter anderem deshalb anonym genannt, weil ihnen – im Gegensatz zu Funktionen, die mit dem Schlüsselwort `def` deklariert werden – niemals ein explizites `__name__`-Attribut übergeben wird.

Currying: teilweise Anwendung von Argumenten

Currying ist Informatiker-Slang (nach dem Mathematiker Haskell Curry) und bedeutet, dass man neue aus existierenden Funktionen ableitet durch die *teilweise Anwendung von Argumenten*. Nehmen wir zum Beispiel an, wir hätten eine triviale Funktion, die zwei Zahlen miteinander addiert:

```
def add_numbers(x, y):
    return x + y
```

Mithilfe dieser Funktion könnten wir eine neue Funktion mit einer Variablen ableiten, `add_five`, die zu ihrem Argument 5 addiert:

```
add_five = lambda y: add_numbers(5, y)
```

Man sagt, das zweite Argument zu `add_numbers` sei *curried*. Das ist hier nichts Besonderes, wir haben einfach nur eine neue Funktion definiert, die eine vorhandene Funktion aufruft. Das eingebaute `functools`-Modul kann diesen Vorgang mit der `partial`-Funktion vereinfachen:

```
from functools import partial
add_five = partial(add_numbers, 5)
```

Generatoren

Ein wichtiges Merkmal in Python ist eine konsistente Methode, über Sequenzen zu iterieren, etwa über Objekte in einer Liste oder Zeilen in einer Datei. Erreicht wird das durch das *Iterator-Protokoll*, eine generische Möglichkeit, Objekte iterierbar zu machen. Zum Beispiel erhält man beim Iterieren über ein Dictionary dessen Schlüssel:

```
In [180]: some_dict = {'a': 1, 'b': 2, 'c': 3}

In [181]: for key in some_dict:
.....:     print(key)
a
b
c
```

Wenn Sie `for key in some_dict` schreiben, versucht der Python-Interpreter zuerst, aus `some_dict` einen Iterator zu erzeugen:

```
In [182]: dict_iterator = iter(some_dict)

In [183]: dict_iterator
Out[183]: <dict_keyiterator at 0x7fbbd5a9f908>
```

Ein Iterator ist jedes Objekt, das Objekte an den Python-Interpreter liefert, wenn es etwa im Kontext einer `for`-Schleife verwendet wird. Die meisten Methoden, die eine Liste oder ein listenartiges Objekt erwarten, akzeptieren auch ein iterierbares Objekt. Dazu gehören eingebaute Methoden wie `min`, `max` und `sum` sowie Typkonstrukturen wie `list` und `tuple`:

```
In [184]: list(dict_iterator)
Out[184]: ['a', 'b', 'c']
```

Ein *Generator* ist eine einfache Möglichkeit, ein neues iterierbares Objekt zu konstruieren. Während normale Funktionen ausgeführt werden und zu einem Zeitpunkt nur ein einziges Ergebnis zurückliefern, liefern Generatoren eine Sequenz von mehreren Werten durch Lazy-Evaluation, wobei sie nach jedem Wert pausieren, bis der nächste gefordert wird. Um einen Generator herzustellen, nutzen Sie in einer Funktion das Schlüsselwort `yield` anstelle von `return`:

```
def squares(n=10):
    print('Generating squares from 1 to {0}'.format(n ** 2))
    for i in range(1, n + 1):
        yield i ** 2
```

Rufen Sie den Generator dann tatsächlich auf, wird zunächst einmal kein Code ausgeführt:

```
In [186]: gen = squares()
```

```
In [187]: gen
```

```
Out[187]: <generator object squares at 0x7fbbd5ab4570>
```

Erst wenn Sie Elemente vom Generator anfordern, beginnt er damit, seinen Code auszuführen:

```
In [188]: for x in gen:
```

```
.....:     print(x, end=' ')
```

```
Generating squares from 1 to 100
```

```
1 4 9 16 25 36 49 64 81 100
```

Generator-Ausdrücke

Eine noch knappere Methode, einen Generator herzustellen, bieten *Generator-Ausdrücke*. Dabei handelt es sich um einen Generator analog zu List, Dict und Set Comprehensions; um einen zu erzeugen, schließen Sie das, was ansonsten eine List Comprehension wäre, in runde statt in eckige Klammern ein:

```
In [189]: gen = (x ** 2 for x in range(100))
```

```
In [190]: gen
```

```
Out[190]: <generator object <genexpr> at 0x7fbbd5ab29e8>
```

Das ist vollständig äquivalent zu dem folgenden deutlich ausführlicheren Generator:

```
def _make_gen():
    for x in range(100):
        yield x ** 2
gen = _make_gen()
```

Generator-Ausdrücke können in vielen Fällen anstelle von List Comprehensions als Funktionsargumente eingesetzt werden:

```
In [191]: sum(x ** 2 for x in range(100))
```

```
Out[191]: 328350
```

```
In [192]: dict((i, i **2) for i in range(5))
```

```
Out[192]: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Das Modul itertools

Das Standardbibliotheksmodul `itertools` besitzt eine Kollektion aus Generatoren für viele gebräuchliche Datenalgorithmen. So nimmt zum Beispiel `groupby` eine Sequenz und eine Funktion entgegen. Der Rückgabewert der Funktion ist das Kriterium, nach dem aufeinanderfolgende Elemente in der Sequenz gruppiert werden. Hier ist ein Beispiel:

```
In [193]: import itertools
```

```
In [194]: first_letter = lambda x: x[0]

In [195]: names = ['Alan', 'Adam', 'Wes', 'Will', 'Albert', 'Steven']

In [196]: for letter, names in itertools.groupby(names, first_letter):
.....:     print(letter, list(names)) # names is a generator
A ['Alan', 'Adam']
W ['Wes', 'Will']
A ['Albert']
S ['Steven']
```

In Tabelle 3-2 sehen Sie eine Liste weiterer `itertools`-Funktionen, die ich oft ganz nützlich finde. Noch mehr Informationen über dieses sinnvolle Hilfsmittel finden Sie außerdem in der offiziellen Python-Dokumentation (<https://docs.python.org/3/library/itertools.html>).

Tabelle 3-2: Einige nützliche `itertools`-Funktionen

Funktion	Description
<code>combinations(iterable, k)</code>	Generiert eine Sequenz aller möglichen <code>k</code> -Tupel von Elementen im iterierbaren Argument, wobei die Reihenfolge ignoriert wird und ohne Ersatz gearbeitet wird (siehe auch die Begleitfunktion <code>combinations_with_replacement</code>).
<code>permutations(iterable, k)</code>	Generiert eine Sequenz aller möglichen <code>k</code> -Tupel von Elementen im iterierbaren Argument unter Beachtung der Reihenfolge.
<code>groupby(iterable[, keyfunc])</code>	Generiert <code>(key, sub-iterator)</code> für jeden eindeutigen Schlüssel.
<code>product(*iterables, repeat=1)</code>	Generiert das kartesische Produkt der iterierbaren Eingangsargumente als Tupel, ähnlich einer verschachtelten <code>for</code> -Schleife.

Fehler und die Behandlung von Ausnahmen

Der kontrollierte Umgang mit Python-Fehlern oder *Ausnahmen* (Exceptions) ist wichtig, wenn man robuste Programme schreiben möchte. Viele Funktionen in Datenanalyseanwendungen funktionieren nur mit bestimmten Eingabearten. Ein Beispiel: Pythons `float`-Funktion ist in der Lage, einen String in eine Gleitkommazahl umzuwandeln, schlägt aber bei falschen Eingaben fehl und gibt einen `ValueError` aus:

```
In [197]: float('1.2345')
Out[197]: 1.2345

In [198]: float('something')
-----
ValueError                                Traceback (most recent call last)
<ipython-input-198-439904410854> in <module>()
----> 1 float('something')
ValueError: could not convert string to float: 'something'
```

Nehmen wir an, wir möchten eine Version von `float` haben, die kontrolliert scheitert und das Eingabeargument zurückliefert. Dazu könnten wir eine Funktion schreiben, die den Aufruf für `float` in einen `try/except`-Block einschließt:

```
def attempt_float(x):
    try:
        return float(x)
    except:
        return x
```

Der Code im except-Teil des Blocks wird nur ausgeführt, wenn float(x) einen Ausnahmefehler verursacht:

```
In [200]: attempt_float('1.2345')
Out[200]: 1.2345

In [201]: attempt_float('something')
Out[201]: 'something'
```

Sie werden vielleicht feststellen, dass float neben ValueError auch andere Ausnahmen auslösen kann:

```
In [202]: float((1, 2))
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-202-842079ebb635> in <module>()
----> 1 float((1, 2))
TypeError: float() argument must be a string or a number, not 'tuple'
```

Sie wollen möglicherweise nur ValueError unterdrücken, da ein TypeError (die Eingabe war weder ein String noch ein numerischer Wert) auf einen ernsthaften Bug in Ihrem Programm hindeuten könnte. Geben Sie dazu nach except die Art der Ausnahme an:

```
def attempt_float(x):
    try:
        return float(x)
    except ValueError:
        return x
```

Wir haben dann:

```
In [204]: attempt_float((1, 2))
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-204-9bdfd730cead> in <module>()
----> 1 attempt_float((1, 2))
<ipython-input-203-3e06b8379b6b> in attempt_float(x)
      1 def attempt_float(x):
      2     try:
----> 3         return float(x)
      4     except ValueError:
      5         return x
TypeError: float() argument must be a string or a number, not 'tuple'
```

Wenn Sie ein Tupel von Ausnahmetypen angeben (die runden Klammern sind obligatorisch), können Sie gleich mehrere Ausnahmetypen erwischen:

```
def attempt_float(x):
    try:
        return float(x)
```

```
except (TypeError, ValueError):
    return x
```

Manchmal wollen Sie eine Ausnahme gar nicht unterdrücken, sondern möchten, dass Code ausgeführt wird, egal ob er im try-Block erfolgreich ist oder nicht. Dazu nutzen Sie `finally`:

```
f = open(path, 'w')

try:
    write_to_file(f)
finally:
    f.close()
```

Hier wird das Datei-Handle `f` *immer* geschlossen. Genauso können Sie Code haben, der nur dann ausgeführt wird, wenn der `try`-Block erfolgreich ist. Dazu verwenden Sie `else`:

```
f = open(path, 'w')

try:
    write_to_file(f)
except:
    print('Failed')
else:
    print('Succeeded')
finally:
    f.close()
```

Ausnahmen in IPython

Wenn ein Ausnahmefehler angezeigt wird, während ein Skript läuft (`%run`) oder eine Anweisung ausgeführt wird, erstellt IPython standardmäßig einen kompletten Stack-trace (Traceback) mit ein paar Zeilen Kontext zu jeder Position in dem Traceback:

```
In [10]: %run examples/ipython_bug.py
-----
AssertionError                                Traceback (most recent call last)
/home/wesm/code/pydata-book/examples/ipython_bug.py in <module>()
     13     throws_an_exception()
     14
----> 15 calling_things()

/home/wesm/code/pydata-book/examples/ipython_bug.py in calling_things()
     11 def calling_things():
     12     works_fine()
----> 13     throws_an_exception()
     14
     15 calling_things()

/home/wesm/code/pydata-book/examples/ipython_bug.py in throws_an_exception()
      7     a = 5
      8     b = 6
----> 9     assert(a + b == 10)
```

```
10
11 def calling_things():
```

AssertionError:

Der zusätzliche Kontext ist ein riesiger Vorteil gegenüber dem normalen Python-Interpreter (der diesen nicht liefert). Sie können die Menge des angezeigten Kontexts mit dem magischen Befehl `%xmode` kontrollieren, von `Plain` (wie der Python-Standard-Interpreter) bis `Verbose` (wo auch die Werte der Funktionsargumente und mehr angezeigt werden). Wie Sie in diesem Kapitel noch erfahren werden, können Sie für ein interaktives Postmortem-Debugging *in den Stack* gehen (mit dem magischen Befehl `%debug` oder `%pdb`), nachdem ein Fehler aufgetreten ist.

3.3 Dateien und das Betriebssystem

In diesem Buch werden üblicherweise abstraktere Tools wie `pandas.read_csv` verwendet, um Daten von der Festplatte in die Python-Datenstrukturen einzulesen. Dennoch ist es wichtig, grundlegend zu verstehen, wie man mit Dateien in Python arbeitet. Das ist zum Glück recht einfach und daher einer der Gründe dafür, dass Python für das Herumfummeln mit Text und Dateien so beliebt ist.

Zum Öffnen einer Datei für das Lesen oder Schreiben nehmen Sie die eingebaute `open`-Funktion mit einem relativen oder absoluten Dateipfad:

```
In [207]: path = 'examples/segismundo.txt'
```

```
In [208]: f = open(path)
```

Die Datei wird standardmäßig schreibgeschützt geöffnet, also im Modus `'r'`. Wir können das Datei-Handle `f` dann wie eine Liste behandeln und über die Zeilen iterieren:

```
for line in f:
    pass
```

Die Zeilen kommen mit intakten Zeilenendmarkierungen (End-of-Line; EOL) aus der Datei, daher werden Sie oft Code sehen, der eine EOL-freie Liste von Zeilen einer Datei erstellt:

```
In [209]: lines = [x.rstrip() for x in open(path)]
```

```
In [210]: lines
```

```
Out[210]:
```

```
['Sueña el rico en su riqueza,',
 'que más cuidados le ofrece;',
 '',
 'sueña el pobre que padece',
 'su miseria y su pobreza;',
 '',
 'sueña el que a medrar empieza,',
 'sueña el que afana y pretende,',
```

```
'sueña el que agravia y ofende,',
'',
'y en el mundo, en conclusión,',
'todos sueñan lo que son,',
'aunque ninguno lo entiende.',
'']
```

Wenn Sie `open` zum Erzeugen von Dateiobjekten benutzen, ist es wichtig, die Datei nach Abschluss der Bearbeitung immer explizit zu schließen. Dadurch werden ihre Ressourcen wieder zurück an das Betriebssystem übergeben:

```
In [211]: f.close()
```

Eine der Möglichkeiten, das Säubern geöffneter Dateien zu erleichtern, bietet die `with`-Anweisung:

```
In [212]: with open(path) as f:
.....:     lines = [x.rstrip() for x in f]
```

Dabei wird die Datei `f` automatisch geschlossen, wenn der `with`-Block verlassen wird.

Hätten wir `f = open(path, 'w')` eingegeben, wäre eine neue Datei unter *examples/segismundo.txt* angelegt worden, die die an dieser Stelle vorhandene Datei überschrieben hätte (seien Sie also vorsichtig). Es gibt außerdem den Dateimodus `'x'`, der eine beschreibbare Datei anlegt, aber fehlschlägt, wenn der Dateipfad bereits existiert. In Tabelle 3-3 sehen Sie eine Liste aller gültigen Lese-/Schreibmodi für Dateien.

Bei lesbaren Dateien sind einige der am häufigsten verwendeten Methoden `read`, `seek` und `tell`. `read` liefert eine bestimmte Anzahl von Zeichen aus der Datei zurück. Was ein »Zeichen« ist, wird durch die Kodierung der Datei (z.B. UTF-8) bestimmt. Beim Öffnen der Datei im Binärmodus bekommt man einfach die reinen Bytes:

```
In [213]: f = open(path)
```

```
In [214]: f.read(10)
Out[214]: 'Sueña el r'
```

```
In [215]: f2 = open(path, 'rb') # Binary mode
```

```
In [216]: f2.read(10)
Out[216]: b'Sue\xc3\xb1a el '
```

Die `read`-Methode schiebt die Position des Datei-Handles um die Anzahl der gelesenen Bytes. `tell` gibt Ihnen die aktuelle Position aus:

```
In [217]: f.tell()
Out[217]: 11
```

```
In [218]: f2.tell()
Out[218]: 10
```

Obwohl wir 10 Zeichen aus der Datei ausgelesen haben, ist die Position 11, da es so viele Bytes gebraucht hat, um mit der Standardkodierung 10 Zeichen zu dekodieren. Sie können die Standardkodierung im Modul `sys` prüfen:

```
In [219]: import sys

In [220]: sys.getdefaultencoding()
Out[220]: 'utf-8'
```

`seek` ändert die Dateiposition auf das angegebene Byte in der Datei:

```
In [221]: f.seek(3)
Out[221]: 3

In [222]: f.read(1)
Out[222]: 'ñ'
```

Und schließlich denken wir daran, die Dateien zu schließen:

```
In [223]: f.close()

In [224]: f2.close()
```

Tabelle 3-3: Python-Dateimodi

Modus	Beschreibung
r	Schreibgeschützter (Read-only-)Modus.
w	Nur-Schreib-Modus; erzeugt eine neue Datei (Daten einer bereits vorhandenen Datei desselben Namens werden gelöscht).
x	Nur-Schreib-Modus; erzeugt eine neue Datei, schlägt aber fehl, falls der Dateipfad bereits existiert.
a	An eine existierende Datei anhängen (erzeugt die Datei, falls sie noch nicht existiert).
r+	Lesen und Schreiben.
b	Bei Binärdateien zum Modus hinzufügen (d.h. 'rb' oder 'wb').
t	Textmodus für Dateien (Bytes werden automatisch nach Unicode dekodiert). Wurde nichts anderes angegeben, ist dies der Standard. Fügen Sie t zu anderen Modi hinzu, um diesen Modus zu benutzen (d.h. 'rt' oder 'xt').

Um Text in eine Datei zu schreiben, können Sie die Methode `write` oder `writelines` benutzen. So könnten wir zum Beispiel eine Version von *prof_mod.py* ohne leere Zeilen anlegen:

```
In [225]: with open('tmp.txt', 'w') as handle:
.....:     handle.writelines(x for x in open(path) if len(x) > 1)

In [226]: with open('tmp.txt') as f:
.....:     lines = f.readlines()

In [227]: lines
Out[227]:
['Sueña el rico en su riqueza,\n',
'que más cuidados le ofrece;\n',
'sueña el pobre que padece\n',
```

```
'su miseria y su pobreza;\n',
'sueña el que a medrar empieza,\n',
'sueña el que afana y pretende,\n',
'sueña el que agravia y ofende,\n',
'y en el mundo, en conclusión,\n',
'todos sueñan lo que son,\n',
'aunque ninguno lo entiende.\n']
```

In Tabelle 3-4 finden Sie viele der gebräuchlichsten Dateimethoden.

Tabelle 3-4: Wichtige Dateimethoden oder -attribute in Python

Methode	Beschreibung
read([size])	Liefert die Daten aus der Datei als String zurück, das optionale size-Argument gibt die Anzahl der zu lesenden Bytes an.
readlines([size])	Liefert die Liste der Zeilen in der Datei zurück, das size-Argument ist optional.
write(str)	Schreibt den übergebenen String in die Datei.
writelines(strings)	Schreibt die übergebene Sequenz aus Strings in die Datei.
close()	Schließt das Handle.
flush()	Schreibt den internen I/O-Puffer auf die Festplatte.
seek(pos)	Geht an die angegebene Dateiposition (Integer).
tell()	Liefert die aktuelle Dateiposition als Integer zurück.
closed	True, falls die Datei geschlossen ist.

Bytes und Unicode mit Dateien

Das normale Verhalten für Python-Dateien (ob lesbar oder schreibbar) ist der *Textmodus*. Das bedeutet, dass Sie planen, mit Python-Strings (d.h. Unicode) zu arbeiten. Dem steht der *Binärmodus* gegenüber, den Sie erreichen, wenn Sie `b` an den Dateimodus anhängen. Schauen wir uns die Datei aus dem vorherigen Abschnitt an (sie enthält Nicht-ASCII-Zeichen mit einer UTF-8-Kodierung):

```
In [230]: with open(path) as f:
.....:     chars = f.read(10)
```

```
In [231]: chars
Out[231]: 'Sueña el r'
```

UTF-8 ist eine Unicode-Kodierung mit variabler Länge. Wenn ich also eine gewisse Anzahl von Zeichen aus einer Datei anfordere, liest Python ausreichend Bytes (das können lediglich 10, aber auch bis zu 40 Bytes sein) aus der Datei, um entsprechend viele Zeichen zu dekodieren. Öffne ich die Datei stattdessen im `'rb'`-Modus, fordert `read` die exakte Anzahl an Bytes an:

```
In [232]: with open(path, 'rb') as f:
.....:     data = f.read(10)
```

```
In [233]: data
Out[233]: b'Sue\xc3\xb1a el '
```

Je nach Textkodierung können Sie die Bytes selbst in ein str-Objekt dekodieren, aber nur wenn alle kodierten Unicode-Zeichen voll ausgeformt sind:

```
In [234]: data.decode('utf8')
Out[234]: 'Sueña el '
```

```
In [235]: data[:4].decode('utf8')
-----
UnicodeDecodeError                                Traceback (most recent call last)
<ipython-input-235-300e0af10bb7> in <module>()
----> 1 data[:4].decode('utf8')
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc3 in position 3: unexpecte
d end of data
```

Der Textmodus bietet in Kombination mit der encoding-Option von open eine bequeme Methode, um eine Unicode-Kodierung in eine andere zu konvertieren:

```
In [236]: sink_path = 'sink.txt'

In [237]: with open(path) as source:
.....:     with open(sink_path, 'xt', encoding='iso-8859-1') as sink:
.....:         sink.write(source.read())

In [238]: with open(sink_path, encoding='iso-8859-1') as f:
.....:     print(f.read(10))
Sueña el r
```

Hüten Sie sich davor, seek zu benutzen, wenn Sie Dateien in einem anderen als dem Binärmodus öffnen. Fällt die Dateiposition in die Mitte der Bytes, die ein Unicode-Zeichen definieren, verursachen die nachfolgenden Lesevorgänge einen Fehler:

```
In [240]: f = open(path)

In [241]: f.read(5)
Out[241]: 'Sueña'
```

```
In [242]: f.seek(4)
Out[242]: 4

In [243]: f.read(1)
-----
UnicodeDecodeError                                Traceback (most recent call last)
<ipython-input-243-7841103e33f5> in <module>()
----> 1 f.read(1)
/miniconda/envs/book-env/lib/python3.6/codecs.py in decode(self, input, final)
    319         # decode input (taking the buffer into account)
    320         data = self.buffer + input
--> 321         (result, consumed) = self._buffer_decode(data, self.errors, final)
    )
    322         # keep undecoded input until the next call
    323         self.buffer = data[consumed:]
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb1 in position 0: invalid s
tart byte

In [244]: f.close()
```

Wenn Sie regelmäßig Nicht-ASCII-Textdaten analysieren, werden Sie merken, dass es sinnvoll ist, die Unicode-Funktionalität von Python zu verstehen. Mehr Informationen dazu finden Sie in der Python-Online-Dokumentation (<https://docs.python.org/>).

3.4 Schlussbemerkung

Wir haben nun die Grundlagen sowie die Python-Umgebung und die Sprache kennengelernt. Wenden wir uns nun NumPy und der Array-orientierten Datenverarbeitung in Python zu.

